EPOCH

INDIVIDUAL WORK ALLOCATION & DELIVERY PLAN

Adithya | 30% of total team workload

Project	EPOCH — Research Integrity & Evidence Workspace
Assigned share	30%
Role	Technical Lead, Integrity Engine & Integration Owner
Architecture baseline	EPOCH_Final_System_Architecture_v2.md (12 November 2026)
Document purpose	Individual execution brief — tasks, deliverables, dependencies and acceptance criteria

Mission

Own the integrity-critical engine foundation and make the separate workstreams fit one coherent, testable EPOCH workflow.

Ownership boundary

Primary owner for the pure-Python epoch-engine contract, canonical data/split logic, structured prediction validation, verdict evaluation, CLI integration and cross-module technical decisions. Coordinates interfaces and integration; does not become the sole implementer of every module.

Workload allocation

This assignment represents 30% of the planned team effort. The percentage is a workload share, not a claim that the work is already complete. Scope changes must be agreed by the team and recorded; do not silently transfer unfinished tasks.

Non-negotiable project rules

- Integrity Core must work end-to-end before optional extensions are treated as deliverables.
- Human researchers author hypotheses and plans; the system records, constrains, evaluates and audits the workflow.
- The sealed confirmation split must not be exposed to exploration, ordinary API responses, or AI features.
- Every confirmatory plan is approved and frozen before execution; each confirmatory plan consumes one unit of the lineage budget K.
- Post-hoc work is labelled and descriptive; it must never be promoted to Supported.
- AI is optional, off by default, and never performs statistical calculations or silently edits a human-owned plan.
- All results must be reproducible and verifiable from a signed evidence bundle.

1. Detailed task breakdown

M1 — Engine contract and repository foundation

- Create/confirm the Python package layout for epoch-engine and define stable public functions: profile, split, validate_prediction, evaluate, verify_bundle and reproduce_bundle.
- Define typed input/output models and deterministic serialization rules. Record engine version as the Git SHA or an explicitly versioned build identifier.
- Write the interface note consumed by the API owner and executor owner: input schemas, output schemas, error codes, deterministic behaviour and artifact conventions.

M2 — Canonical dataset profiling and lineage/split logic

- Implement canonical CSV parsing/profile interfaces in coordination with the executor owner; define allowed parsing settings and reject unsupported or ambiguous inputs.
- Implement deterministic canonical row representation and stable row keys. Implement deployment-secret HMAC-based split assignment; never return or log the secret salt.
- Implement sticky assignment for edited rows using stable identifiers or documented nearest-neighbour matching; preserve prior assignment for matched confirmation rows.
- Implement containment-based lineage matching using normalized per-column value fingerprints (architecture threshold ≥0.8), tenant-scoped lineage boundaries and explicit uncertainty/hold decisions.
- Support committed split-membership bitmap/hash at freeze and the 'holdout revealed' lock after export. Expose only safe confirmation profile metadata (n, column types, missing counts).

M3 — Structured predictions and verdict evaluator

- Define and validate machine-checkable predictions: test, outcome, predictors, covariates, direction, SESOI, alpha, missing-data handling and declared variants.
- Implement the first-match verdict rules exactly as defined in the v2 architecture; return exactly one verdict for every valid result and make unsupported/invalid cases Inconclusive or explicit errors as specified.
- Enforce post-hoc restrictions: a plan/check created after a confirmatory result cannot produce Supported.
- Implement property/golden tests across dense estimate-confidence-interval cases and edge cases such as assumption failure, missing columns and invalid parameters.

M4 — CLI, reproducibility and integration

- Implement or coordinate epoch-engine CLI commands: run, verify, reproduce and bench entry points, with the core run/verify/reproduce path prioritised.
- Define deterministic result.json and bundle manifest formats with hashes, environment lock and audit-head references. Validate bundle tampering and clean-machine reproduction.
- Integrate the engine with the backend and isolated executor through versioned interfaces; review contract changes before they are merged.
- Run an end-to-end vertical slice: investigation → data → exploration → hypothesis → frozen plan → isolated run → verdict → report/bundle.

2. Required deliverables

Every item below must be produced, committed or linked, and demonstrated. A verbal update alone is not a deliverable.

- Engine package skeleton, documented public API and versioned schemas.
- Canonical profiling and deterministic HMAC split/lineage implementation, including sticky assignment and holdout-export lock.
- Structured prediction validator and first-match verdict evaluator.
- Golden-value/property tests and negative tests for leakage, post-hoc plans and malformed inputs.
- CLI verify/reproduce path and signed-bundle verification integration.
- Integration demo and an engineering handoff note explaining contracts, run commands and unresolved risks.

3. Definition of done and acceptance criteria

A deliverable is accepted only when the relevant evidence exists (tests, API contract, migration, screenshot plus working route, reproducible command, or review record).

- Re-uploading the same CSV in a different row order or with a renamed filename preserves split membership and lineage budget.
- Editing a confirmation row does not silently move it into exploration; dropping 15% of rows or renaming/reordering columns is handled by the v2 lineage rules.
- The HMAC salt never appears in API responses, UI, logs or bundles; exported membership bitmap matches the freeze commitment and prevents further confirmatory plans on that lineage.
- Every valid evaluator input produces exactly one deterministic verdict; later runs do not rewrite earlier verdicts.
- Plans created after a confirmatory result are post-hoc and cannot be Supported.
- Tampered bundles fail verification; clean-machine reproduction produces the same result for the same pinned environment.

4. Dependencies and handoffs

- Needs dataset/question, outcome, ID/group/time columns and parse settings agreed by the team (M0).
- Needs Sushma's API/data-model contract before engine endpoints are wired.
- Needs Atul's executor result schema, isolation boundary and job lifecycle.
- Pranav consumes the stable API response and report/bundle schema; do not make the UI depend on internal Python objects.

5. Explicitly out of scope

- Do not build the complete website alone or own all API routes.
- Do not implement arbitrary user code execution, a graph database, adaptive holdout reuse or organisation-wide workspaces.
- Do not claim statistical truth, causal identification or security certification.

6. Milestones and sequencing

Follow the architecture's M0–M7 build order. The dates are intentionally not invented here; the team should set calendar dates after M0 confirms the dataset, question, capacity and technical feasibility.

Milestone	Your expected contribution
M0 — Decide	Contribute to dataset/question selection, define outcome and test catalogue, confirm owners and K policy.
M1 — Engine	Deliver profile/split, four Core tests, evaluator, golden tests and bundle verify/reproduce.
M2 — Backend	Stabilise engine/API schemas and integrate one isolated confirmatory run.
M3 — UI slice	Support a complete user journey using stable interfaces.
M4–M6 — Extensions/hardening	Review extension contracts and help validate benchmark/security evidence; do not let extensions block Core.

7. Required weekly progress update

- Completed: list merged work and link the PR/commit, migration, test report or document.
- In progress: identify the exact task, current status and next concrete action.
- Blocked: name the dependency, who can unblock it and the decision needed.
- Evidence: provide command output, test result, API response, short recording or reproducible demo.
- Risk/change: state any scope change, security concern, schema change or estimate change before acting on it.

8. Collaboration and handoff checklist

- Agree interface/schema changes with dependent owners before implementation; avoid parallel incompatible assumptions.
- Open a focused pull request with a clear summary, test evidence and known limitations.
- Include setup/run instructions and sample configuration without secrets.
- Never commit credentials, signing keys, dataset secrets or personal/regulated data.
- Demonstrate the feature from a clean checkout or documented environment before marking it complete.

9. Review 1 contribution

For Review 1, present the product integrity problem, system boundaries, engine-first build order, split/lineage threat model, verdict rules and a small reproducible CLI demonstration if ready. Clearly label unimplemented items.

10. Shared project completion standard

- All code is committed to the agreed repository and the team can run it from documented setup instructions.
- Interfaces, request/response schemas, database migrations and environment variables are documented.
- Unit tests and integration tests are included; no task is considered complete on the basis of a screenshot alone.
- Known limitations, assumptions, risks and deferred features are recorded rather than hidden.
- Changes are reviewed by the relevant module owner and integrated through the shared branch/PR process.

11. Architecture traceability

This work plan is derived from the consolidated v2 architecture. Primary sections relevant to this assignment: §3 Scope tiers; §4 System architecture; §6 Website journey; §7 Integrity Core; §8 Extensions; §9 Data model; §10 Engine interface and test catalogue; §11 AI guardrails; §12 Security/privacy/reliability; §13 Evaluation/benchmark; §14 Build order; §15 Acceptance criteria; §16 Risks; §17 Open decisions; §21 v2 corrections. The individual plan does not override the architecture.

Important status note: the architecture is a proposed design, not evidence of implementation, load testing or security certification. Each owner must report actual implementation status honestly.